

编码心得分享

前言

是分享，是讨论，不是培训；

从 talk is cheap，show me the code，到 code is cheap，show me the talk

（今天的讨论既有 code，也有 talk）

AI时代不需要讨论具体某个方法的代码怎么优化，纯动动“嘴” talk，即可获得高质量 code，甚至没手都行。

会发展成为纯 体力 or 脑力 劳动？两极化的趋势加剧。

讨论编码不是目的，是我们达到 **降本增效** 目的的手段。

（解放生产力、提升个人价值，将有限的精力投入到更有价值的事情中，人的价值与事情的价值是相互成就的。）

做正确的事 or 正确的做事

“Done is better than perfect”，如何理解 done？

软件开发三原则

make it work **FIRST**

make it right THEN

make it fast **FINALLY**

如何理解其中的 right？

- 1 const right
- 2 var right

“Make it work” 是第一原则

只有在“Make it work”的基础上，才有资格讨论“Make it right”和“Make it fast”。有些人会问：“为什么不能直接从 right 开始？”这就像“第七个包子”的故事——既然吃七个包子就能饱，为什么不直接吃第七个呢？

理论上可以一开始就做对，但实际很难，因为“正确”本身是动态变化的。

“正确”的三大决定因素

1. 认知范围：

你的学习会不断接触新理论、新工具。例如，从直连数据库到 Connection Pool，再到 ORM，甚至缓存优化，每一种技术都会刷新“正确”的定义。

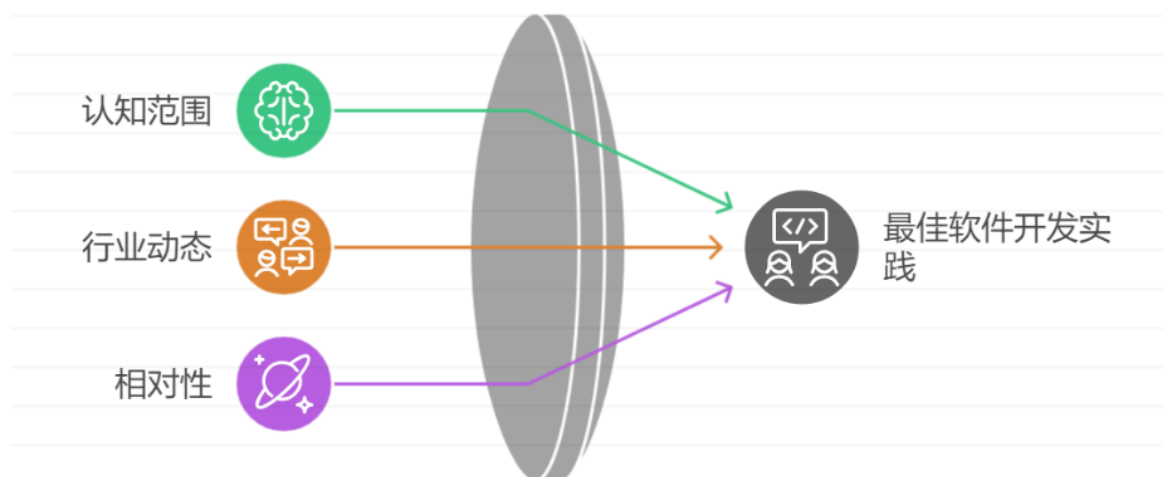
2. 行业动态：

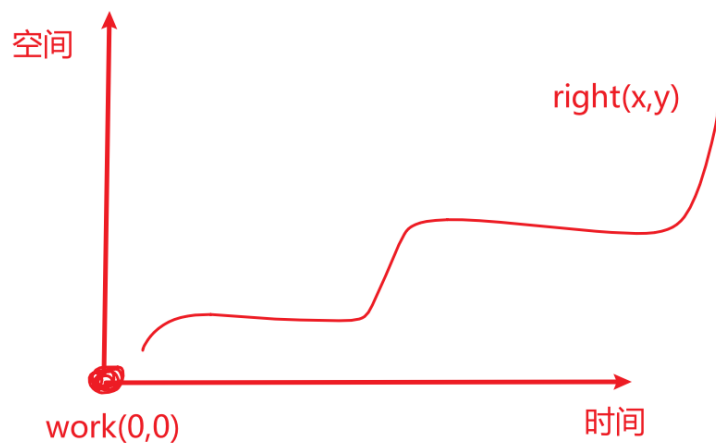
当前的“正确”往往是阶段性的。例如，前端技术更新频繁，不同社区对技术的接受度也不同。

3. 相对性：

软件开发没有绝对的“正确”或“错误”，只有“更好”和“更差”。追求“足够好”即可，过度完美只会拖延整体进度。

驾驭软件开发的复杂性





案例1 - 报告生成

以基于 poi-tl 实现的报告生成为例，算不算 done ？

取决于看待这件事的角度

案例2 - 框架定时调度

现状

目标

效果

缺陷

未来

干中学

建议的理念：干中学 与 费曼学习法，以及建构主义的学习理论不谋而合。

一下扎入细节当中去，容易碰壁受挫，打击积极性，或迷失方向；

没有亲身实践落地，没有经过生产检验的成果，基本是纸上谈兵。



建构主义（Constructivism）是一种关于知识和学习的理论，强调学习者在学习过程中主动建构知识，而不是被动接受信息。建构主义认为，知识不是通过教师传授直接获得的，而是学习者在一定的社会文化背景下，借助他人的帮助，利用必要的学习资料，通过意义建构的方式获得的。

建构主义学习方式的核心观点

- **主动建构**：学习者不是被动地接收信息，而是主动地建构知识。他们将新信息与已有的知识经验相结合，形成新的理解。
- **情境性**：学习是在特定的情境中发生的，知识的学习和应用是紧密相关的。学习者需要在真实或模拟的情境中进行学习，以便更好地理解和应用知识。
- **社会互动**：学习是一个社会过程，学习者通过与他人的互动来建构知识。合作学习、讨论和交流是建构主义学习的重要组成部分。
- **反思**：学习者需要对自己的学习过程进行反思，思考自己的理解和假设，从而不断调整和深化知识。

建构主义学习方式的应用

- **探究式学习**：鼓励学生提出问题、设计实验、收集数据并得出结论。这种方法强调学生的主动参与和探索。
- **项目式学习**：通过完成一个具体的项目来学习知识和技能。项目通常涉及真实世界的问题，需要学生综合运用多学科知识。
- **合作学习**：学生分组合作，共同完成任务。通过讨论和协作，学生可以分享不同的观点和想法，促进知识的建构。
- **情境教学**：将学习内容置于真实或模拟的情境中，使学生能够在实际应用中学习和理解知识。
- **反思性实践**：鼓励学生对学习过程进行反思，思考自己的学习策略和方法，从而不断改进学习效果。

建构主义学习方式的优势

- **提高学习动机**：学生在主动参与和探索中获得成就感，提高学习兴趣和动机。
- **促进深度理解**：通过主动建构知识，学生能够更深入地理解和掌握知识。
- **培养批判性思维**：学生在探究和反思中发展批判性思维能力，学会独立思考和解决问题。
- **增强合作能力**：通过合作学习，学生学会与他人沟通和协作，提高团队合作能力。

总之，建构主义学习方式强调学习者的主体地位，鼓励学生主动探索、合作交流和反思实践，从而实现知识的深度建构和有效应用。



细节

细节不重要的前提：细节已做的足够好。

对其中的“不重要”的理解，认为绝大多数人能够做好，门槛较低，而不是做不到也不重要；
细节就是，当你做到，正确的不那么明显；但是没做到，就会错误的非常突出。

语言

language & code，作为语言 language 最最最最最首要的目的是人容易理解、交流。

《其实你是一位作家》

纯code更像语言学科，是语言就有特定的、规范的组织模式。

解构 + 重构，“形而上者谓之道，形而下者谓之器”

本质是采用一种语法、逻辑更严谨，并有特定语言组织技巧的语言来描述客观世界。

（特定算法除外，但此类问题99%借助工具、开源，务必不要自己造轮子）

读一篇作文，怎么分辨出是小学生流水账 or 高中生组织严密的应试议论文 or 公务员申论的八股公文？

主题模糊

语言匮乏

缺乏技巧

逻辑跳跃

（组织论点-需求与目标、论据-工具与理论、论证-实践与实现，剩下的就是怎么润色、连接这些素材）

code smell & code taste，教一个小学生所有高考满分作文的技巧，也是吸收不了的。

以后 poi-tl 为例，这篇“作文”该打多少分？

文档

去文档化的思路，什么是文档最需要记录的核心内容？

直接阅读源码，难以理解的最大痛点是什么？

（what？how？why！代码、方案的高频变动，文档无法维护，而且文档天然存在细节信息容易缺失的弊端。）

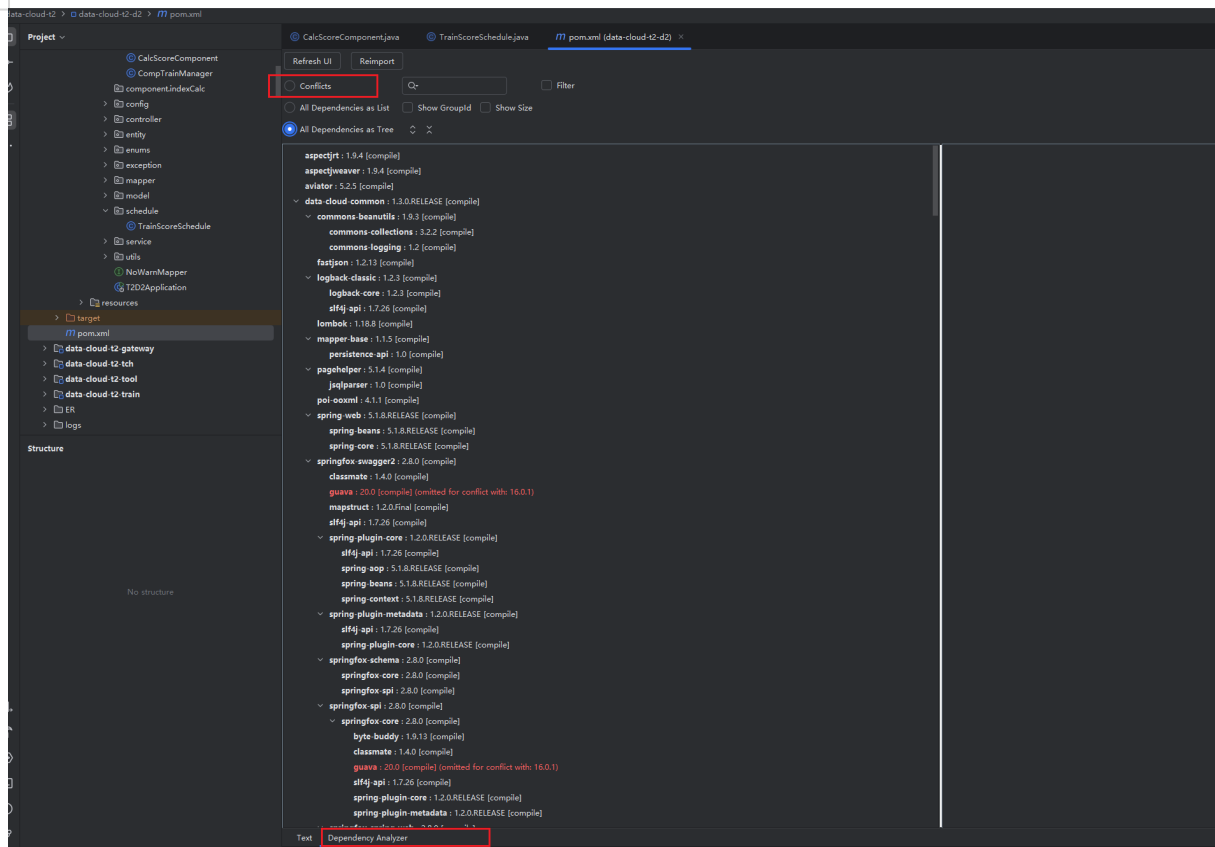
软件核心的复杂性，并不是来源于庞大的软件体量或者复杂的业务流程，而是来自于项目长期迭代过程中，不断冒出的超出当初设计之外的不确定性。

关于 owner 意识

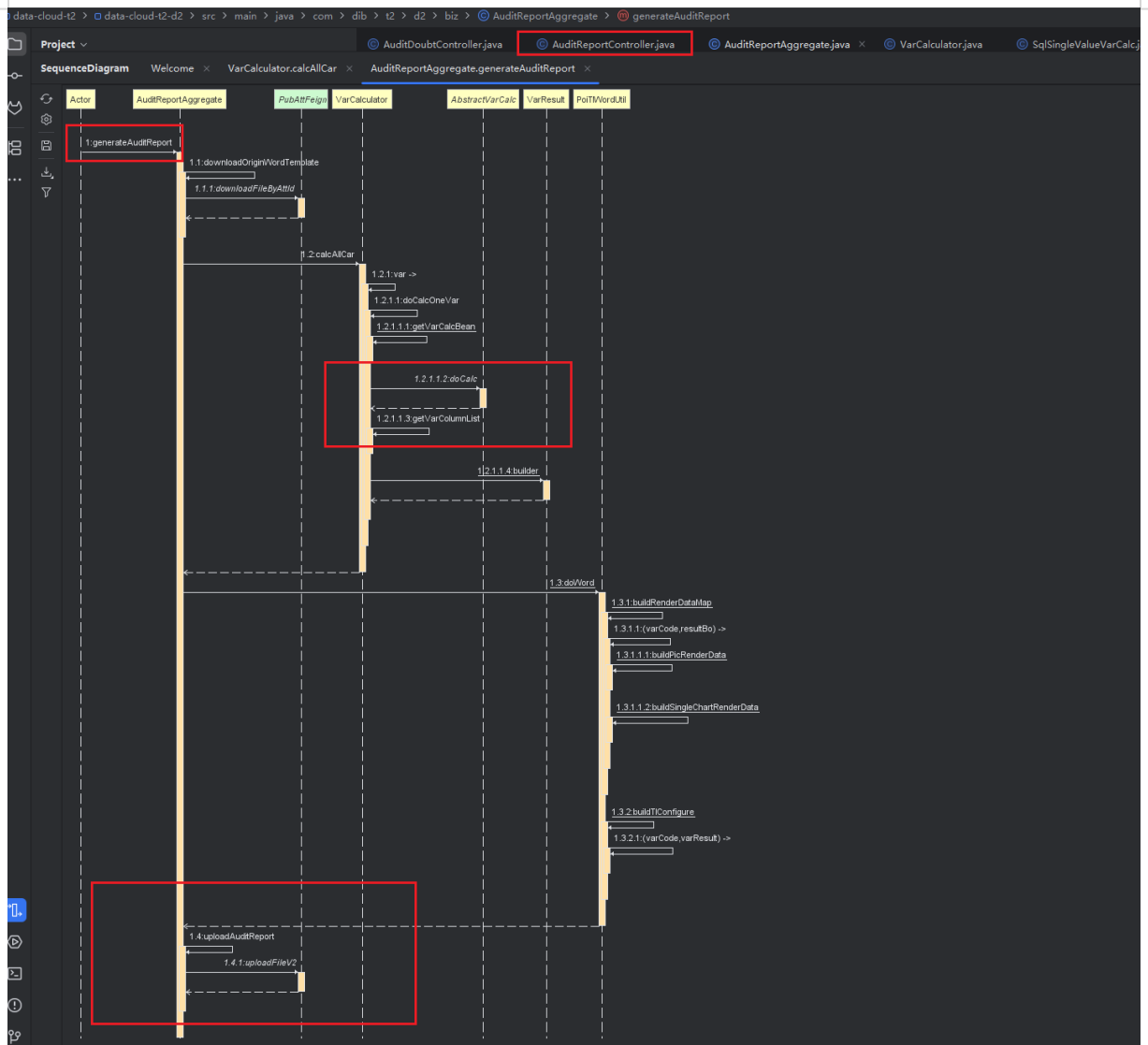
code 层面能力期望：腐败（初中级）or 保持（高级）or 演进（资深）

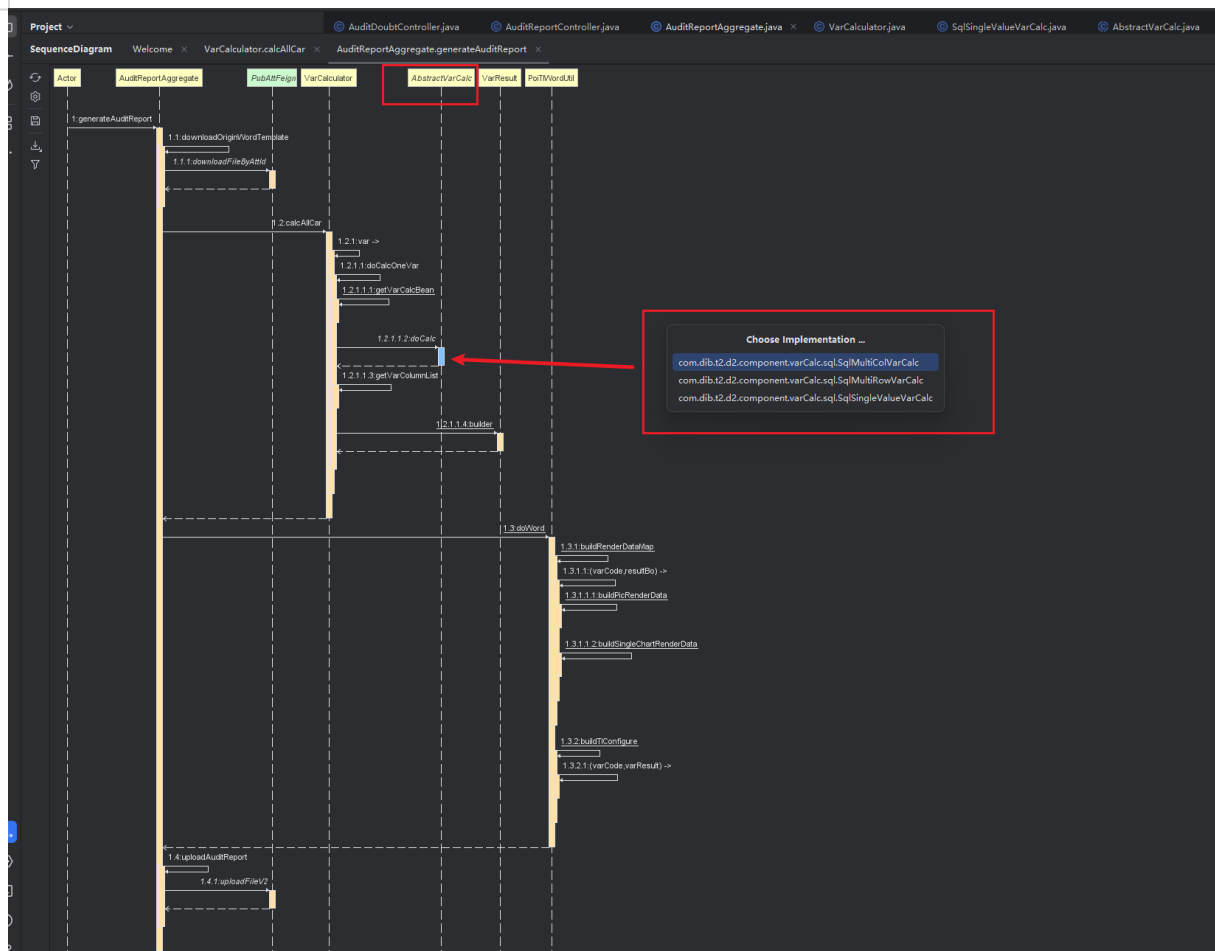
工程工具

maven helper



sequence diagram





docker、arthas、elk、apm工具

代码工具

多交流、思考，借鉴更高效的实践技巧。

guava、hutool 中的集合工具

Returns every possible list that can be formed by choosing one element from each of the given sets in order; the "n-ary Cartesian product" of the sets. For example:

```
Sets.cartesianProduct(ImmutableList.of(
    ImmutableSet.of(1, 2),
    ImmutableSet.of("A", "B", "C")))
```

returns a set containing six lists:

- `ImmutableList.of(1, "A")`
- `ImmutableList.of(1, "B")`
- `ImmutableList.of(1, "C")`
- `ImmutableList.of(2, "A")`
- `ImmutableList.of(2, "B")`
- `ImmutableList.of(2, "C")`

The result is guaranteed to be in the "traditional", lexicographical order for Cartesian products that you would get from nesting for loops:

```
for (B b0 : sets.get(0)) {
    for (B b1 : sets.get(1)) {
        ...
        ImmutableList<B> tuple = ImmutableList.of(b0, b1, ...);
        // operate on tuple
    }
}
```

Note that if any input set is empty, the Cartesian product will also be empty. If no sets at all are provided (an empty list), the resulting Cartesian product has one element, an empty list (counter-intuitive, but mathematically consistent).

枚举工具

```
216 }
217
218 通过 某字段对应值 获取 枚举，获取不到时为 null
219 Params: enumClass - 枚举类
220 predicate - 条件
221 Returns: 对应枚举，获取不到时为 null
222 Since: 5.8.0
223
224 public static <E extends Enum<E>> E getBy(Class<E> enumClass, Predicate<? super E> predicate) {
225     return Arrays.stream(enumClass.getEnumConstants())
226         .filter(predicate).findFirst().orElse(null);
227 }
228
229 通过 某字段对应值 获取 枚举，获取不到时为 null
230 Params: condition - 条件字段
231 value - 条件字段值
232 Returns: 对应枚举，获取不到时为 null
233
234 public static <E extends Enum<E>, C> E getBy(Func1<E, C> condition, C value) {
235     Class<E> implClass = LambdaUtil.getRealClass(condition);
236     if (Enum.class.equals(implClass)) {
237         implClass = LambdaUtil.getRealClass(condition);
238     }
239     return Arrays.stream(implClass.getEnumConstants()).filter(e -> condition.callWithRuntimeException(e).equals(value)).findAny().orElse(null);
240 }
241
242 通过 某字段对应值 获取 枚举，获取不到时为 defaultEnum
243 Params: condition - 条件字段
244 value - 条件字段值
245 defaultEnum - 条件找不到时返回结果使用这个
246 Returns: 对应枚举，获取不到时为 null
247 Since: 5.8.8
248
249 public static <E extends Enum<E>, C> E getBy(Func1<E, C> condition, C value, E defaultEnum) {
250     return ObjectUtil.defaultIfNull(getBy(condition, value), defaultEnum);
251 }
252 }
```

IO工具

```
}

int read;
byte[] bytes = new byte[16 * 1024];
try (InputStream input = response.body().asInputStream();
    OutputStream out = targetResponse.getOutputStream()) {
    while ((read = input.read(bytes)) >= 0) {
        out.write(bytes, off: 0, read);
    }
}

Decompiled .class file, bytecode version: 52.0 (Java 8)
D:\maven\repository\dib\com\google\guava\guava\33.2.0-jre\guava-33.2.0-jre.jar\com\google\

144 public final class ByteStreams {
145     private static byte[] combineBuffers(Queue<byte[]> bufs, int totalLen) {
154         int bytesToCopy;
155         for (result = Arrays.copyOf(result, totalLen); remaining > 0; remaining -= bytesToCopy) {
156             byte[] buf = (byte[]) bufs.remove();
157             bytesToCopy = Math.min(remaining, buf.length);
158             int resultOffset = totalLen - remaining;
159             System.arraycopy(buf, srcPos: 0, result, resultOffset, bytesToCopy);
160         }
161         return result;
162     }
163 }
164 }
165 }
166 }

167 @ public static byte[] toByteArray(InputStream in) throws IOException {
168     Preconditions.checkNotNull(in);
169     return toByteArrayInternal(in, new ArrayDeque< numElements: 20, totalLen: 0);
170 }
171 }

172 @ static byte[] toByteArray(InputStream in, long expectedSize) throws IOException {
173     Preconditions.checkArgument(b: expectedSize >= 0L, errorMessageTemplate: "expectedSize (%s) must be non-negative", expectedSize);
174     if (expectedSize > 2147483639L) {
175         throw new OutOfMemoryError(s: expectedSize + " bytes is too large to fit in a byte array");
176     } else {
177         byte[] bytes = new byte[(int) expectedSize];
178
179         int read;
180         for (int remaining = (int) expectedSize; remaining > 0; remaining -= read) {
181             int off = (int) expectedSize - remaining;
182             read = in.read(bytes, off, remaining);
183             if (read == -1) {
184                 return Arrays.copyOf(bytes, off);
185             }
186         }
187     }
}
```

命名

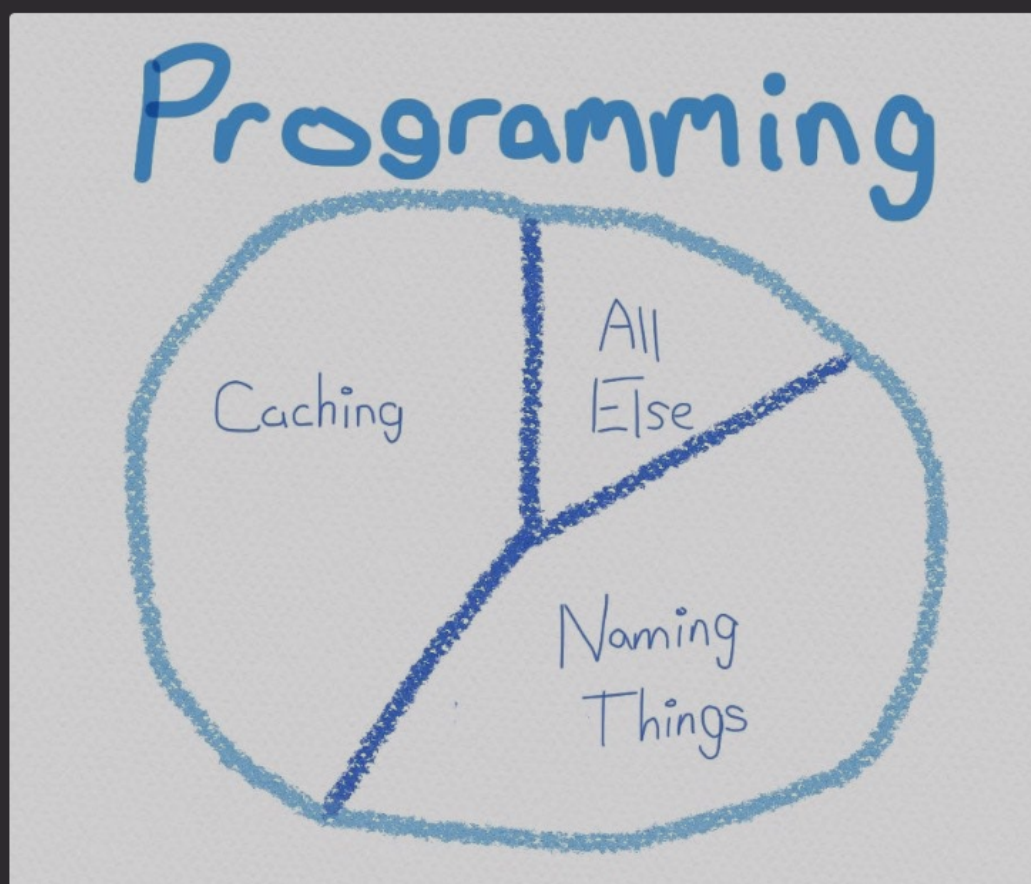
(包括但不限于, 类名、包名、工程名等等)

There are only two hard things in computer science: cache invalidation and naming things.

总结
参考

计算机科学中只有两件难事：缓存失效和命名。

—— (Phil Karlton) 菲尔·卡尔顿



命名重要且难

正如Stack Overflow的创始人Joel Spolsky所说：

“Creating good names is hard, but it should be hard, because a great name captures essential meaning in just one or two words.”（起一个好名字应该很难，因为，一个好名字需要把要义浓缩在一到两个词中。）

命名为什么难呢？因为命名的过程本身就是一个抽象和思考的过程，在工作中，当我们不能给一个模块、一个对象、一个函数，甚至一个变量找到合适的名称的时候，往往说明我们对问题的理解还不够透彻，需要重新去挖掘问题的本质，对问题域进行重新分析和抽象，有时还要调整设计和重构代码。因此，好的命名是我们写出好代码的基础。

因此，要想真正做好命名，除了要掌握一些命名技法，还需要更深层次的修炼——提升抽象思维能力。

以下总结命名错误的几个层次、原因及规避方法。

Donald Knuth 曾经说过: "Programs are meant to be ready by humans and only incidentally for computers to execute." 代码是用来让人读的, 只是顺便让机器执行而已。(原话或许是 Hal Abelson 说的)



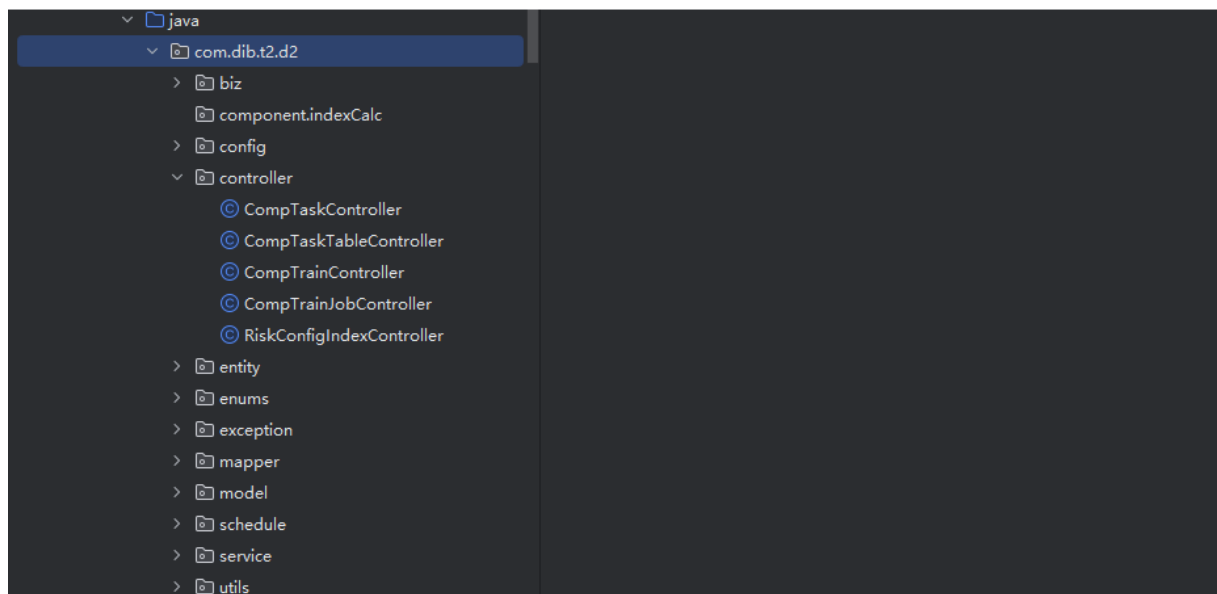
Programs are meant to be read by
humans and only incidentally for
computers to execute.

— Donald Knuth —

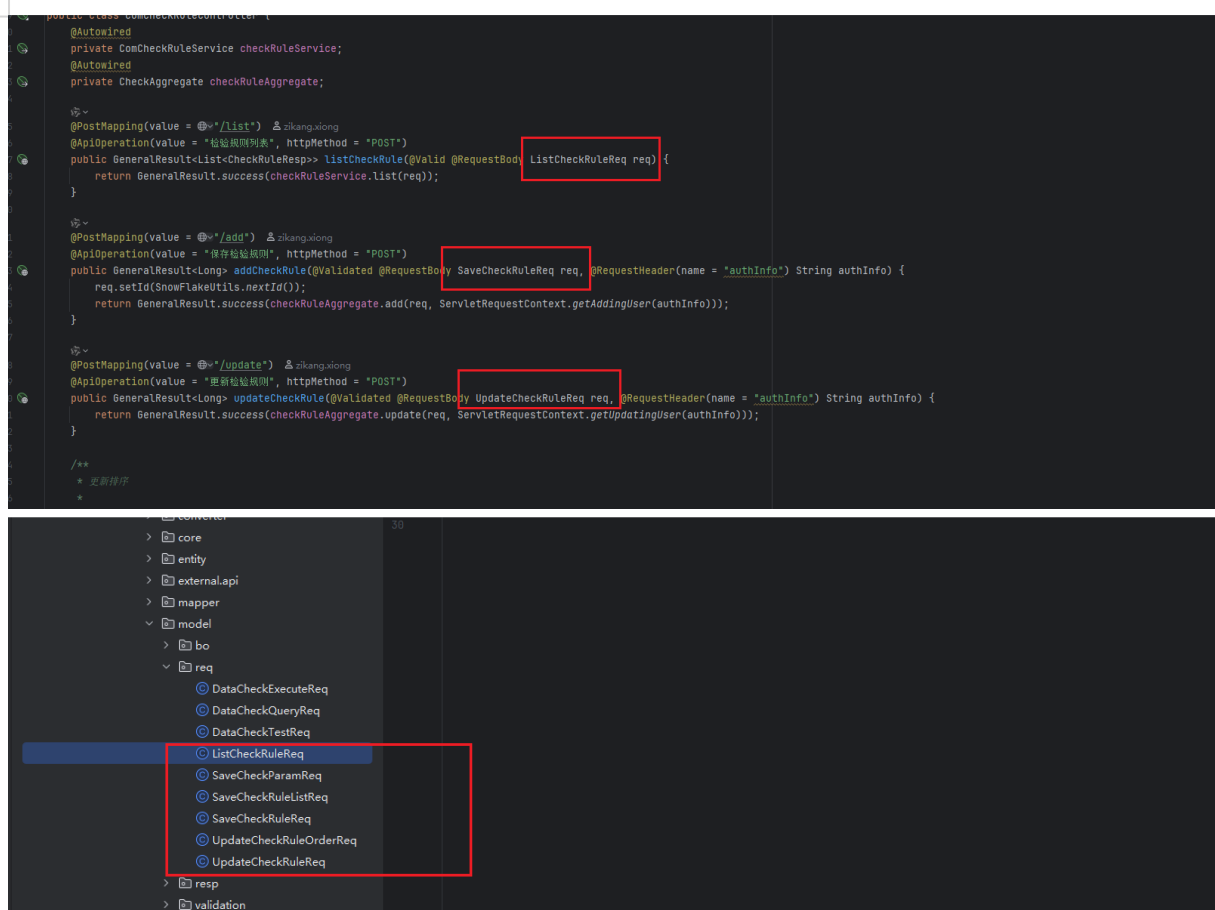
AZ QUOTES

代码示例 - package的重要性

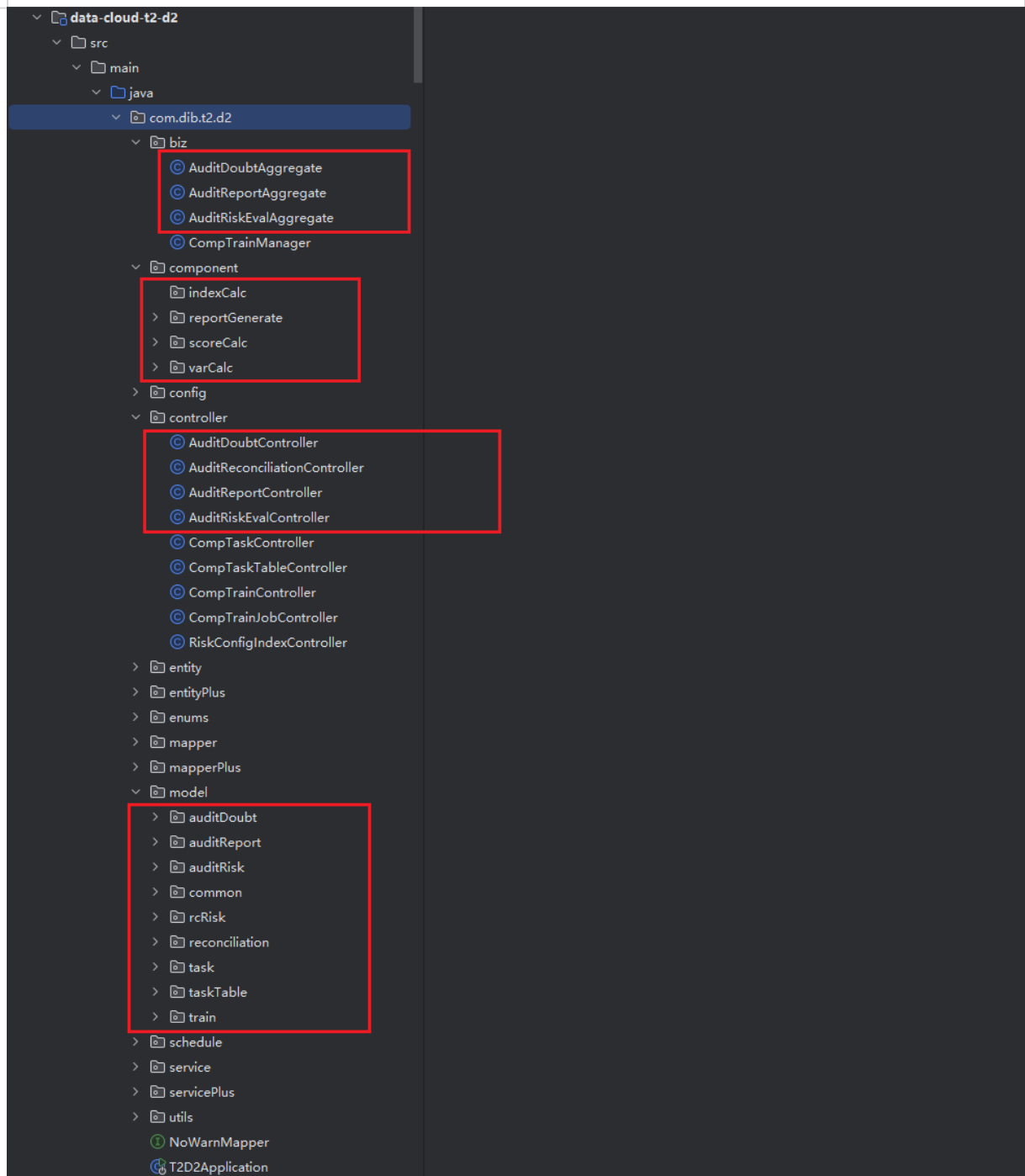
改之前



- model
 - bo
 - CommonPage
 - query
 - BizDataQuery
 - CompRiskCustomerQuery
 - TrainBizQuery
 - request
 - BaseRequest
 - BusinessAttributeListReq
 - CalculateTestReq
 - CompRiskCustomerReq
 - CompRiskCustomerSaveReq
 - CompScoreReq
 - CompTableDataReq
 - CorrectTableDataReq
 - IndexInfoListReq
 - IndexInfoSaveReq
 - IndexListReq
 - PageParam
 - PageReq
 - TableAttSaveReq
 - TableDataImportReq
 - TaskAttSaveReq
 - TaskUserReq
 - ThresholdItem
 - response
 - CompRiskCustomerResp
 - CompRiskDataMateResp
 - CompRiskDataResp
 - CompTableResponse
 - CompTaskMonitorResponse
 - CompTaskOrgResp
 - CompTaskResponse
 - D2TableColumnResp
 - IndexListResp
 - RcspBusAttrInfoResp
 - TableDataInfoResponse
 - train
 - trainjob
 - OrgTrainBo
 - OrgTrainDeleteReq
 - OrgTrainPublishReq
 - OrgTrainSaveReq
 - TrainJobAddReq
 - TrainJobDetailResp
 - TrainJobListQuery
 - TrainJobListQuery



改之后



代码示例 - 风险计算

为什么不喜欢阅读他人的代码
为什么拒绝复用他人写的代码
为什么不要轻易的自己造轮子
改之前

```

@Override Image @RequestMapping
public List<CompRiskDataMaterResp> getRiskCustomerDataList(CompRiskCustomerQuery query) {
    List<CompRiskDataMaterResp> returnList = new ArrayList<>();
    Long trainId = query.getTrainId();
    try {
        int status = compScoreService.checkIsPublish(trainId, query.getTaskId(), query.getUpdateBy(), CaseItemEnum.FRAMEWORK_CONSTRUCTION.getCode());
        if (0 == status) {
            // 获取等级MAP
            Map<String, Integer> riskLevelMap = new HashMap<>();

            // 1、获取当前任务列表
            List<CompRiskCustomerReq> customerList = getRiskCustomerList(query);
            Map<String, List<CompRiskCustomerReq>> customerMap = customerList.stream().collect(Collectors.groupingBy(CompRiskCustomerReq c -> c.getRiskId() + "-" + c.getFactorCode()));
            // 2、TODO 获取任务ID, userId 匹配的公司列表 - 遍历循环
            List<CompCompanyInfo> companyList = companyInfoMapper.selectByTaskId(query.getTaskId());
            // 3、获取公司ID、任务ID、userId 匹配的指标信息、指标值、指标编号、构建map
            Map<String, List<CompIndexData>> indexDataMap = compIndexDataService.getListByTaskIdAndUserId(query.getUpdateBy(), query.getTaskId(), trainId)
                .stream().collect(Collectors.groupingBy(CompIndexData::getCompanyId));
            for (CompCompanyInfo company : companyList) {
                if (indexDataMap.containsKey(company.getCompanyId())) {
                    // 获取当前公司的指标值
                    List<CompIndexData> indexDataList = indexDataMap.get(company.getCompanyId());
                    Map<String, CompIndexData> map = indexDataList.stream().collect(Collectors.toMap(CompIndexData data -> data.getIndexCode(), CompIndexData o -> risk, (CompIndexData o, CompIndexData n) -> n));
                    // 遍历指标值是否匹配
                    for (String riskIdKey : customerMap.keySet()) {
                        Map<String, List<CompRiskCustomerReq>> riskCustomerMap = customerMap.get(riskIdKey);
                        // 存储单个风险的值-风险与因子之间为或关系、因子与对应的指标为且关系
                        List<CompRiskDataMaterResp> returnOneList = new ArrayList<>();
                        String riskLevelMapKey = "";
                        Integer riskLevelMapValue = 999;

                        for (CompRiskCustomerReq riskCustomer : riskCustomerMap.get(riskIdKey)) {
                            Integer riskLevel = 0;
                            if (map.containsKey(riskCustomer.getIndexCode())) {
                                CompIndexData compIndexData = map.get(riskCustomer.getIndexCode());
                                if (StrUtil.isBlank(compIndexData.getValue())) {
                                    continue;
                                }
                                riskLevel = IntervalCheckerUtils.isValueInInterval(Double.parseDouble(compIndexData.getValue()), riskCustomer.getThresholdItem().getHighThreshold()) ?
                                    1 : IntervalCheckerUtils.isValueInInterval(Double.parseDouble(compIndexData.getValue()), riskCustomer.getThresholdItem().getMediumThreshold()) ?
                                    2 : IntervalCheckerUtils.isValueInInterval(Double.parseDouble(compIndexData.getValue()), riskCustomer.getThresholdItem().getLowThreshold()) ? 3 : 0;
                                if (0 != riskLevel) {
                                    CompRiskDataMaterResp compRiskDataMaterResp = new CompRiskDataMaterResp();
                                    BeanUtils.copyProperties(compIndexData, compRiskDataMaterResp);
                                    BeanUtils.copyProperties(riskCustomer, compRiskDataMaterResp);
                                    compRiskDataMaterResp.setIndexDataValue(compIndexData.getValue());
                                    compRiskDataMaterResp.setRiskLevel(riskLevel);
                                    // todo 填充字段
                                    compRiskDataMaterResp.setCompanyName(company.getCompanyName());
                                    compRiskDataMaterResp.setRiskCustomerId(riskCustomer.getId());
                                    returnOneList.add(compRiskDataMaterResp);
                                    riskLevelMapKey = getIndexDataKey(compRiskDataMaterResp);
                                    riskLevelMapValue = Math.min(riskLevel, riskLevelMapValue);
                                }
                            }
                        }
                        if (0 == riskLevel) {
                            riskLevelMapValue = 0;
                            returnOneList.clear();
                            break;
                        }
                    }
                    returnList.addAll(returnOneList);
                    if (riskLevelMapValue != 0) {
                        riskLevelMap.put(riskLevelMapKey, riskLevelMap.containsKey(riskLevelMapKey) ? Math.min(riskLevelMapValue, riskLevelMap.get(riskLevelMapKey)) : riskLevelMapValue);
                    }
                }
            }

            // 清除旧数据
            compRiskDataMapper.deleteByTaskIdAndUserId(trainId, query.getTaskId(), query.getUpdateBy());

            if (CollUtil.isEmpty(returnList)) {
                List<CompRiskData> riskDataList = new ArrayList<>();
                // 遍历处理输出数据
                returnList.forEach(CompRiskDataMaterResp e -> {
                    e.setRiskLevel(riskLevelMap.get(getIndexDataKey(e)));

                    // 生成新数据入库
                    CompRiskData cr = new CompRiskData();
                    BeanUtils.copyProperties(e, cr);
                    cr.setIdx(null);
                    riskDataList.add(cr);
                });
                this.compRiskDataMapper.insertList(riskDataList);
            }
        } else {
            List<CompRiskDataMaterResp> list = compRiskDataMapper.getListByTaskIdAndUserId(trainId, query.getTaskId(), query.getUpdateBy());
            returnList = MyBeanUtils.copyList(list, CompRiskDataMaterResp::new, (CompRiskDataResp s, CompRiskDataMaterResp t) -> {
                });
        }
    }
    if (CollUtil.isEmpty(returnList)) {
        Map<Long, CompRisk> riskMap = compRiskService.getListByIdSet(returnList.stream().map(CompRiskDataMaterResp::getRiskId).collect(Collectors.toSet()))
            .stream().collect(Collectors.toMap(CompRisk::getId, CompRisk risk -> risk, (CompRisk o, CompRisk n) -> n));
        Map<String, CompIndex> indexMap = compIndexService.getListByCodeSet(returnList.stream().map(CompRiskDataMaterResp::getIndexCode).collect(Collectors.toSet()).stream()
            .collect(Collectors.toMap(CompIndex::getIndexCode, CompIndex risk -> risk, (CompIndex o, CompIndex n) -> n));
    }
}

```

```

        // 生成数据入库
        CompRiskData cr = new CompRiskData();
        BeanUtils.copyProperties(e, cr);
        cr.setId(null);
        riskDataList.add(cr);
    });
    this.compRiskDataMapper.insertList(riskDataList);
}
} else {
    List<CompRiskDataResp> list = compRiskDataMapper.getListByTaskIdAndUserId(trainId, query.getTaskId(), query.getUpdateBy());
    returnList = MyBeanUtils.copyList(list, CompRiskDataMateResp::new, (CompRiskDataResp s, CompRiskDataMateResp t) -> {
    });
}
if (CollUtil.isEmpty(returnList)) {
    Map<Long, CompRisk> riskMap = compRiskService.getListByIdSet(returnList.stream().map(CompRiskDataMateResp::getRiskId).collect(Collectors.toSet()))
        .stream().collect(Collectors.toMap(CompRisk::getId, CompRisk risk -> risk, (CompRisk o, CompRisk n) -> n));
    Map<String, CompIndex> indexMap = compIndexService.getListByCodeSet(returnList.stream().map(CompRiskDataMateResp::getIndexCode).collect(Collectors.toSet()))
        .stream().collect(Collectors.toMap(CompIndex::getCode, CompIndex risk -> risk, (CompIndex o, CompIndex n) -> n));
    Map<String, CompFactor> factorMap = compFactorService.getListByCodeSet(returnList.stream().map(CompRiskDataMateResp::getFactorCode).collect(Collectors.toSet()))
        .stream().collect(Collectors.toMap(CompFactor::getCode, CompFactor risk -> risk, (CompFactor o, CompFactor n) -> n));

    // 循环处理输出数据
    returnList.forEach(CompRiskDataMateResp e -> {
        e.setFactorName(getValueFromObject(factorMap.get(e.getFactorCode()), CompFactor::getFactorName));
        e.setIndexName(getValueFromObject(indexMap.get(e.getIndexCode()), CompIndex::getIndexName));
        e.setRiskName(getValueFromObject(riskMap.get(e.getRiskId()), CompRisk::getRiskName));
    });
}
} catch (Exception e) {
    log.info(">>>>查询匹配结果error:{}, e.getMessage());
}
}
return returnList;
}

```

改之后

```

/**
 * 风险模型计算
 * 若已提交，则抛出异常；
 * 若未提交，则实时计算。
 */
@Transactional 2 usages 2 hurui
public void calcOneRiskModel(ExamineeB0 examineeB0) {
    CaseItemUtil.validatePublishStatus(CaseItemEnum.FRAMEWORK_CONSTRUCTION, examineeB0);

    // 查询风险模型配置
    List<T2CompRiskCustomerEntity> riskCustomerList = riskCustomerServicePlus.lambdaQuery()
        .eq(T2CompRiskCustomerEntity::getTrainId, examineeB0.getTrainId())
        .eq(T2CompRiskCustomerEntity::getTaskId, examineeB0.getCaseId())
        .eq(T2CompRiskCustomerEntity::getUpdateBy, examineeB0.getUserId().toString())
        .list();

    // 查询指标值
    Map<String, String> indexCodeToValueMap = riskScoreUtil.getIndexCodeToValueMap(examineeB0);

    List<T2CompRiskDataEntity> riskDataList = new ArrayList<>(initialCapacity: 10);
    riskCustomerList.forEach(T2CompRiskCustomerEntity riskCustomer -> {
        T2CompRiskDataEntity riskData = new T2CompRiskDataEntity();
        riskData.setId(SnowFlakeUtils.nextId());
        riskData.setRiskCustomerId(riskCustomer.getId());
        // 指标值
        String indexValue = indexCodeToValueMap.get(riskCustomer.getIndexCode());
        riskData.setIndexDataValue(indexValue);
        // 风险等级
        IndexThresholdB0 indexThresholdB0 = JSON.parseObject(riskCustomer.getIndexThreshold(), IndexThresholdB0.class);
        int riskLevel = RiskScoreUtil.getRiskLevel(indexThresholdB0, indexValue);
        riskData.setRiskLevel(riskLevel);
        riskData.setTrainId(examineeB0.getTrainId());
        riskData.setTaskId(examineeB0.getCaseId());
        riskData.setUpdateBy(examineeB0.getUserId().toString());
        riskData.setUpdateTime(new Date());
        // add
        riskDataList.add(riskData);
    });

    // 先删除
    riskDataServicePlus.lambdaUpdate()
        .eq(T2CompRiskDataEntity::getTrainId, examineeB0.getTrainId())
        .eq(T2CompRiskDataEntity::getTaskId, examineeB0.getCaseId())
        .eq(T2CompRiskDataEntity::getUpdateBy, examineeB0.getUserId().toString())
        .remove();

    // 再保存
    riskDataServicePlus.saveBatch(riskDataList);
}

```

```

public List<AuditRiskDataResp> listRiskData(ExamineeB0 examineeB0) {
    // 风险模型计算数据
    List<T2CompRiskDataEntity> riskDataList = riskDataServicePlus.lambdaQuery()
        .eq(T2CompRiskDataEntity::getTrainId, examineeB0.getTrainId())
        .eq(T2CompRiskDataEntity::getTaskId, examineeB0.getCaseId())
        .eq(T2CompRiskDataEntity::getUpdateBy, examineeB0.getUserId().toString())
        .list();

    if (CollectionUtil.isEmpty(riskDataList)) {
        return Collections.emptyList();
    }

    return this.convertRiskDataEntityToResp(examineeB0, riskDataList);
}

```

代码示例 - 风险对比

改之前

```

public class CompRiskCustomerServiceImpl implements CompRiskCustomerService {
    public void saveRiskData(CompRiskCustomerQuery query, List<CompRiskDataMateResp> resplist) {
        String riskLevelMapKey = "";
        int riskLevelMapValue = 999;

        for (RiskCustomerB0 riskCustomer : riskCustomerMap.get(riskIdKey)) {
            int riskLevel = 0;
            if (indexCodeToDataOfOneCompanyMap.containsKey(riskCustomer.getIndexCode())) {
                CompIndexData compIndexData = indexCodeToDataOfOneCompanyMap.get(riskCustomer.getIndexCode());
                if (StringUtil.isBlank(compIndexData.getValue())) {
                    // 没有指标值，跳过
                    log.warn("【风险模型】获取指标值失败，指标编码 = {}", riskCustomer.getIndexCode());
                    continue;
                }

                // 根据指标值 & 阈值位置，来判断风险等级
                riskLevel = RiskScoreUtil.getRiskLevel(riskCustomer.getThresholdItem(), compIndexData.getValue());

                if (0 != riskLevel) {
                    // 指标值匹配到阈值位置
                    CompRiskDataMateResp compRiskDataMateResp = new CompRiskDataMateResp();
                    BeanUtils.copyProperties(compIndexData, compRiskDataMateResp);
                    BeanUtils.copyProperties(riskCustomer, compRiskDataMateResp);
                    compRiskDataMateResp.setIndexDataValue(compIndexData.getValue());
                    compRiskDataMateResp.setRiskLevel(riskLevel);
                    // 公司名称
                    compRiskDataMateResp.setCompanyName(company.getCompanyName());
                    // 模型配置主键ID
                    compRiskDataMateResp.setRiskCustomerId(riskCustomer.getId());
                    // add
                    returnOneList.add(compRiskDataMateResp);
                    riskLevelMapKey = this.getIndexDataKey(compRiskDataMateResp);
                    riskLevelMapValue = Math.min(riskLevel, riskLevelMapValue);
                }
            }

            // todo 这里未下断：没看懂
            if (0 == riskLevel) {
                riskLevelMapValue = 0;
                returnOneList.clear();
                break;
            }
        }

        // todo 没看懂
        resplist.addAll(returnOneList);

        // todo 没看懂
        if (riskLevelMapValue != 0) {
            riskLevelMap.put(riskLevelMapKey, riskLevelMap.containsKey(riskLevelMapKey) ? Math.min(riskLevelMapValue, riskLevelMap.get(riskLevelMapKey)) : riskLevelMapValue);
        }
    }

    // 清除旧数据
    compRiskDataMapper.deleteByTaskIdAndUserId(query.getTrainId(), query.getTaskId(), query.getUpdateBy());

    if (CollectionUtil.isEmpty(resplist)) {

```

改之后

```

/**
 * 风险模型配置的计算结果，与标准答案对比
 */
// 查询学生答案
public List<AuditRiskDataCompareResp> compareRiskData(Examinee80 examinee80) {
    // 查询学生答案
    List<T2CompRiskDataEntity> userRiskDataList = riskDataServicePlus.lambdaQuery()
        .eq(T2CompRiskDataEntity::getTrainId, examinee80.getTrainId())
        .eq(T2CompRiskDataEntity::getTaskId, examinee80.getCaseId())
        .eq(T2CompRiskDataEntity::getUpdateBy, examinee80.getUserId().toString())
        .list();
    List<AuditRiskDataResp> riskDataRespList = this.convertRiskDataEntityToResp(examinee80, userRiskDataList);
    Map<String, AuditRiskDataResp> userKeyToRespMap = riskScoreUtil.convertRiskDataKeyToRespMap(riskDataRespList);

    // 查询标准答案
    T2CompTaskEntity taskEntity = taskServicePlus.getById(examinee80.getCaseId());
    List<T2CompRiskCustomerEntity> correctRiskCustomerList = riskCustomerServicePlus.lambdaQuery().eq(T2CompRiskCustomerEntity::getDataId, taskEntity.getCorrectRiskDataId()).list();
    Set<Long> collectRiskCustomerId = correctRiskCustomerList.stream().map(T2CompRiskCustomerEntity::getId).collect(Collectors.toSet());
    List<AuditRiskDataResp> correctRiskDataList = riskDataServicePlus.lambdaQuery().in(T2CompRiskDataEntity::getRiskCustomerId, collectRiskCustomerId).list();
    List<AuditRiskDataResp> correctRiskDataRespList = this.convertRiskDataEntityToResp(examinee80, correctRiskDataList);
    Map<String, AuditRiskDataResp> correctKeyToRespMap = riskScoreUtil.convertRiskDataKeyToRespMap(correctRiskDataRespList);

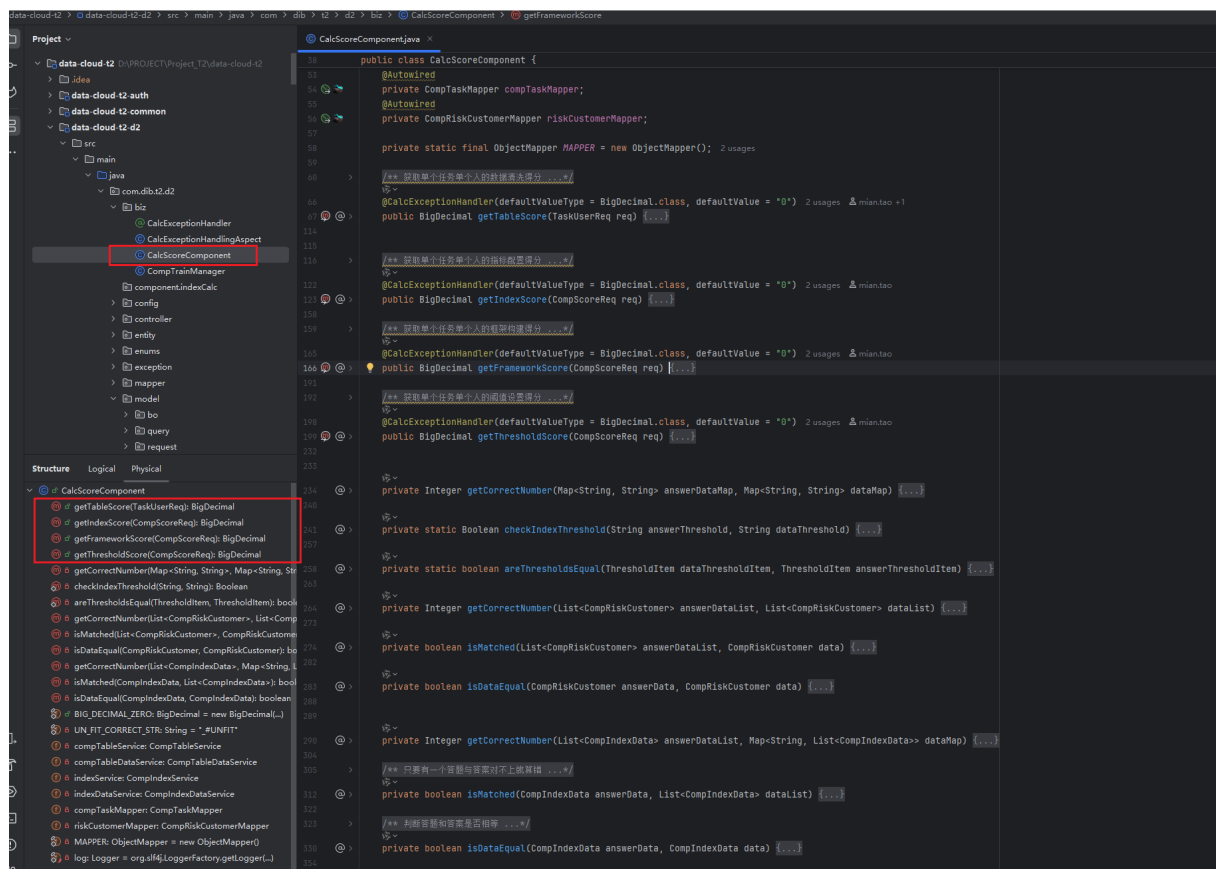
    // 根据 riskId + factorCode + indexCode 作为 riskDataKey 来对比答案
    List<AuditRiskDataCompareResp> respList = new ArrayList<>();
    userKeyToRespMap.forEach((String riskDataKey, AuditRiskDataResp userResp) -> {
        // 如果还有剩余答案，则填充
        if (CollectionUtil.isEmpty(correctKeyToRespMap)) {
            correctKeyToRespMap.values().forEach(AuditRiskDataResp correctResp -> {
        }
    }
    return respList;
}

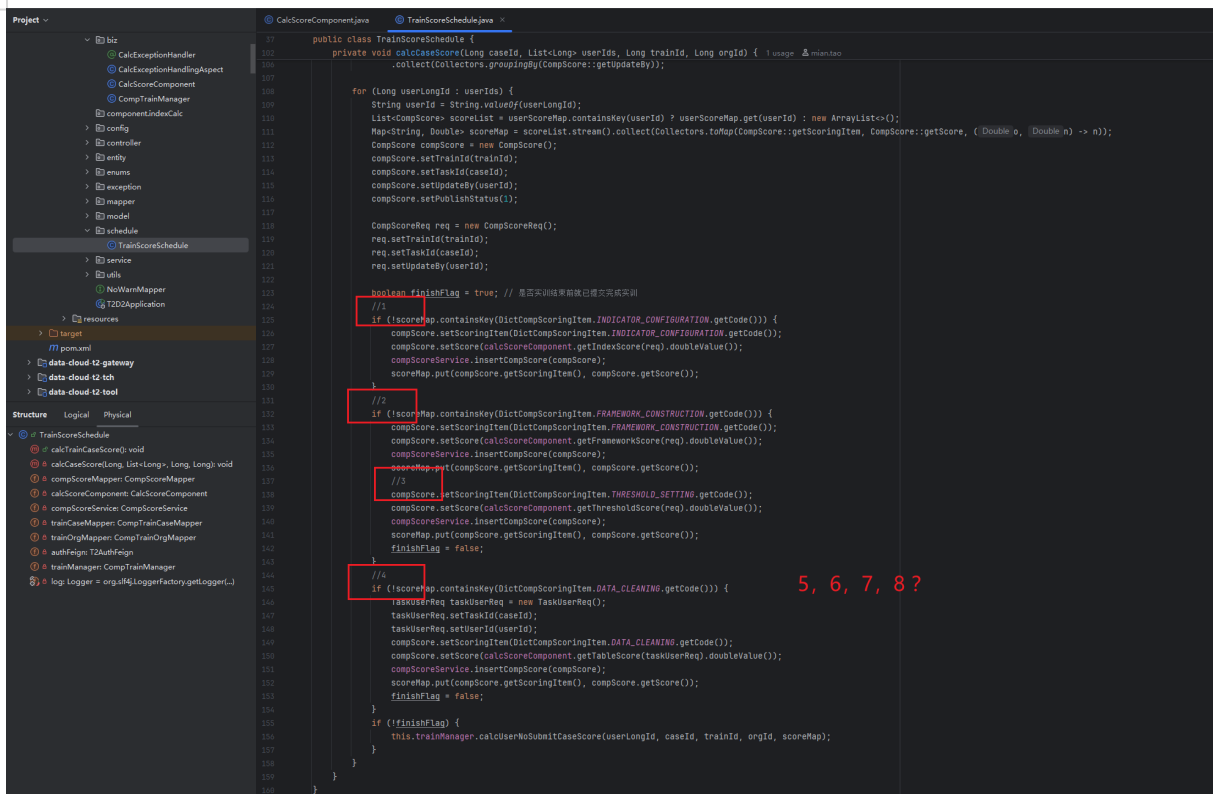
```

代码示例 - 分数计算

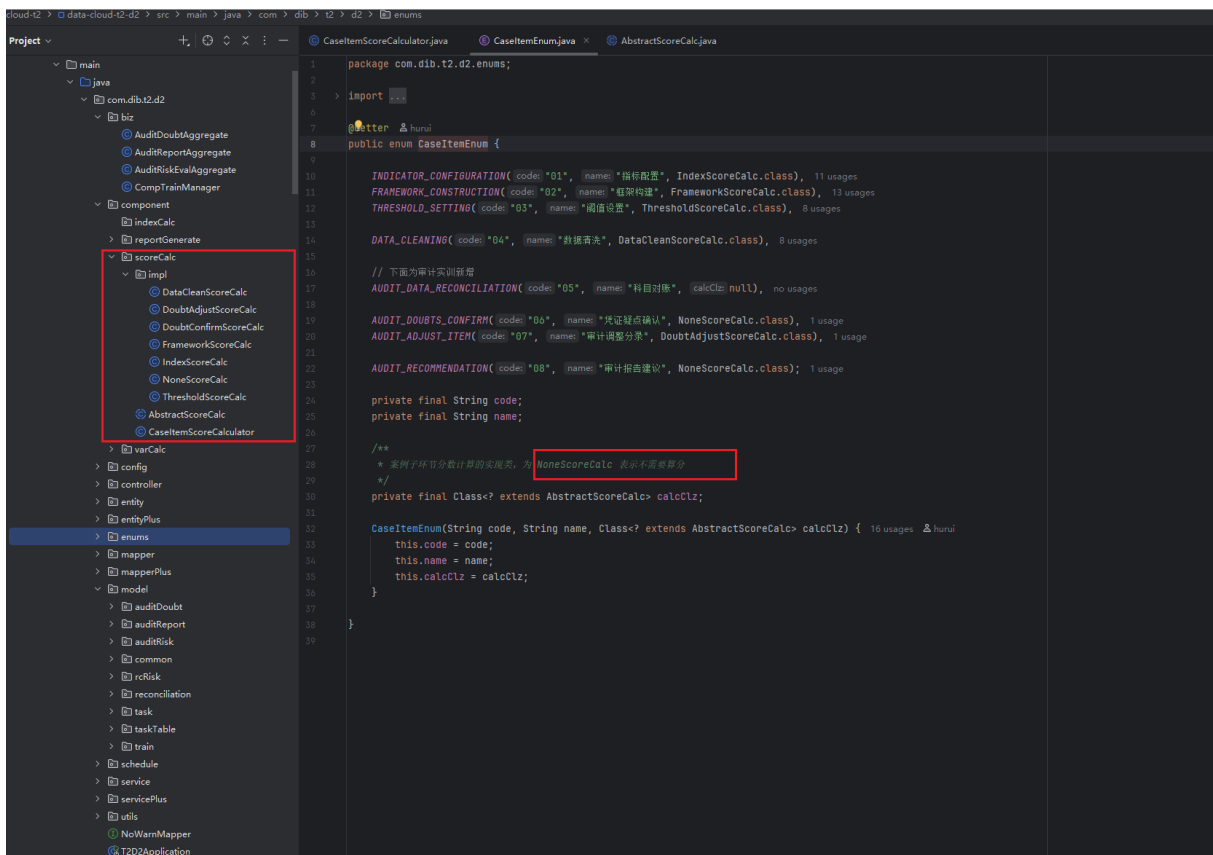
代码天然有自解释属性。

改之前





改之后



代码示例 - 报告生成

以 poi-tl 为例，TDO模式

T (定义模板)

D (准备数据)

O (获得输出)

代码示例

```
SwaggerParser swaggerParser = new SwaggerParser();
Swagger swagger = swaggerParser.read("https://petstore.swagger.io/v2/swagger.json");
SwaggerView viewData = convert(swagger); ❶

LoopRowTableRenderPolicy loopRowTableRenderPolicy = new LoopRowTableRenderPolicy();
Configure config = Configure.builder()
    .bind("parameters", hackLoopTableRenderPolicy)
    .bind("responses", hackLoopTableRenderPolicy)
    .bind("properties", hackLoopTableRenderPolicy)
    .bind("definitionCode", new HighlightRenderPolicy())
    .useSpringEL()
    .build(); ❷

XWPFTemplate template = XWPFTemplate.compile("swagger.docx", config).render(viewData); ❸
template.writeToFile("out_example_swagger.docx");
```

- ❶ 解析Swaggerjson
- ❷ 配置模板引擎
- ❸ Swagger导出Word

没错，一切都是如此简洁，简洁的导出代码，简洁的Word模板，甚至生成的Swagger文档看起来那么简洁，愿一切如你所愿。



源码参见 [JUnit SwaggerToWordExample](#)